SQA Interview Cheat Sheet

Testing Theory • SDLC • STLC • Coverage Criteria • TDD • Defect Lifecycle

Muhammad Faizan | CS3046 Software Testing | FAST-NUCES 2026

1. WHY SQA — YOUR BACKGROUND & MOTIVATION

If asked: Why QA? Why not dev?

"During my CS degree at FAST-NUCES I built full-stack and AI projects — but I kept noticing that building software without a structured testing mindset leads to fragile, unreliable systems. My Software Testing course (CS3046) under Dr. Usman Ghous taught me formal test design — TDD, coverage criteria, and automated testing with JUnit5. That clicked for me. I realized quality is not an afterthought — it is engineering."

Your Academic Foundation

CS3046 — Software Testing (FAST-NUCES)

- TDD — Test Driven Development (Red-Green-Refactor)
- JUnit5 — automated unit testing in Java
- Black-box: input space partitioning, coverage criteria
- White-box: CFG, node/edge/path coverage
- Logic coverage: predicate, clause, MCDC variants
- Data-flow coverage: all-defs, all-uses
- Integration testing, state machine testing

Supporting Courses

- Software Engineering — SDLC, requirements, BRS/SRS
- Software Design & Analysis — UML, design patterns
- OOP in Java — foundation for JUnit5 frameworks
- Database Systems — SQL for DB testing knowledge
- Web Programming — understanding what you are testing

CGPA: 3.30/4.0 — FAST-NUCES

2. TESTING TYPES — KNOW EVERY ONE

Type	What it is	Who does it
Unit Testing	Test individual functions/classes in isolation	Developer / QA
Integration Testing	Test interaction between modules	QA
System Testing	Test complete application end-to-end	QA
Acceptance Testing	Validate against business requirements	Client / QA
Smoke Testing	Quick check — is the build testable at all? Day 0 check	QA
Sanity Testing	After bug fix — does that specific fix work?	QA
Regression Testing	Re-run all tests after any code change	QA / Automation
Exploratory Testing	Unscripted, experience-based testing	QA
Performance Testing	How fast / stable under load	QA / Perf Eng

Load Testing	Behaviour under expected user load	QA
Stress Testing	Push beyond limits — find breaking point	QA
Security Testing	Find vulnerabilities, injection, auth issues	Security QA
UAT	User Acceptance Testing — real users validate	Client
Alpha Testing	Internal testing before release	Internal team
Beta Testing	External users test before public launch	Real users
Retesting	Test the specific bug that was fixed	QA
Ad hoc Testing	Random testing with no plan or docs	QA
Compatibility	Works across browsers, OS, devices	QA
Usability	Is the UI easy to use?	UX / QA
Accessibility	Can disabled users use it? WCAG compliance	QA

3. BLACK BOX / WHITE BOX / GREY BOX

	Black Box	White Box	Grey Box
Knowledge	No code access	Full code access	Partial code access
Focus	Input/Output behaviour	Internal logic/paths	Both
Who	QA testers	Developers / QA	QA with some code access
Techniques	EP, BVA, decision tables	CFG, path/branch coverage	Mix of both
Example	Login form testing	Testing every if/else path	API + some internals

4. BLACK BOX — INPUT SPACE PARTITIONING

Equivalence Partitioning (EP)

Divide inputs into groups (partitions) where all values in a group are expected to behave the same. Test ONE value from each partition.

Partition	Example (Age field, valid 18-60)	Test value
Valid partition	18 to 60	30
Invalid — too low	Below 18	10
Invalid — too high	Above 60	70
Invalid — non-numeric	Letters	abc

Boundary Value Analysis (BVA)

Test at the EDGES of each partition — bugs hide at boundaries.

Boundary	Values to test
Just below minimum	17
At minimum	18

Just above minimum	19
Just below maximum	59
At maximum	60
Just above maximum	61

Coverage Criteria for Combinations

Criteria	What it means
Each Choice (EC)	At least one value from each partition tested
Pair-wise (2-wise)	Every PAIR of parameter values covered at least once
T-wise	Every combination of T parameters covered
Base Choice (BC)	Pick a base value for each param, vary one at a time
Multiple Base Choice	Multiple base values, vary one at a time each
Combinatorial (All Comb)	Every possible combination — exhaustive, often too many

5. WHITE BOX — CFG & STRUCTURAL COVERAGE

Control Flow Graph (CFG)

CFG Components

- Node — a statement or block of statements
- Edge — control flow between nodes
- Decision node — if/while/for (branching)
- Entry node — start of the method
- Exit node — end of the method

Coverage Criteria (in order of strength)

- Node (Statement) Coverage — every node visited
- Edge (Branch) Coverage — every edge traversed
- Edge-pair Coverage — every consecutive pair of edges
- Prime Path Coverage — every maximal simple path
- Complete Path Coverage — every possible path (often infinite)

Touring

- Direct tour — path goes through node/edge directly
- Side-trip — detour allowed but returns to path
- Detour — any path through the graph

Subsumption (stronger subsumes weaker)

Complete Path > Prime Path > Edge-pair > Edge > Node

Prime Path is called 'one ring to rule them all' in your course

Prime Path Coverage

A prime path is a simple path (no repeated nodes) that is maximal — cannot be extended further.

Prime Path Coverage requires every prime path to be toured. It subsumes edge-pair, edge, and node coverage.

Your course calls it 'the one ring to rule them all' — if you achieve prime path coverage, you automatically satisfy all weaker criteria.

6. DATA FLOW COVERAGE

Key Terms

Term	Meaning	Example
def	Variable is defined (assigned a value)	x = 5
use (c-use)	Variable used in a computation	y = x + 1

use (p-use)	Variable used in a predicate (condition)	if (x > 0)
def-use pair	A def of x followed by a use of x with no redef in between	(def at n1, use at n5)
def-clear path	Path from def to use with no re-definition of variable	No x = something in between

Criteria	Requirement
All-defs	Every def of every variable reaches at least one use
All-uses	Every def-use pair must be covered (all paths from each def to each use)
All-def-use-pairs	Every def-use pair on every def-clear path

Subsumption: All-def-use-pairs > All-uses > All-defs

7. LOGIC COVERAGE CRITERIA

Key Terms

- Predicate — the full boolean expression: (A && B) || C
- Clause — an atomic boolean condition: A, B, C each separately
- Major clause — the clause being tested for independence
- Minor clause — the other clauses

RIPR Model (for a fault to be detected)

- Reachability — test must reach the fault
- Infection — fault must cause wrong state
- Propagation — wrong state must reach output
- Revealability — tester must observe it

Criteria	Abbreviation	Requirement
Predicate Coverage	PC	Every predicate evaluates to T and F
Clause Coverage	CC	Every clause evaluates to T and F
Combinatorial Coverage	CoC	Every combination of clause truth values
Active Clause Coverage	ACC	Major clause independently affects predicate outcome
GACC	General ACC	Major clause determines predicate; minor clauses can be anything
CACC	Correlated ACC	Minor clauses set so predicate is T when major=T and F when major=F
RACC	Restricted ACC	Same minor clause values used for both major=T and major=F tests
GICC	General ICC	Inactive — major clause does NOT affect predicate
RICC	Restricted ICC	Inactive with same minor clause values for both tests

CACC (Correlated ACC) is closest to MCDC used in aviation/safety-critical testing. Your course covers this as 'Active Clause Coverage and its variants'.

8. TDD — TEST DRIVEN DEVELOPMENT

Red-Green-Refactor Loop

RED	GREEN	REFACTOR
Write a failing test first. No production code exists yet. Test fails — that is correct.	Write the MINIMUM code needed to make the test pass. Do not over-engineer.	Clean up the code. Remove duplication. Tests must still pass.

Why TDD matters

- Forces you to think about design before code
- Every feature has a test by default
- Bugs found at write-time, not release-time
- Enables safe refactoring
- Living documentation — tests describe behaviour
- Reduces debugging time significantly
- Supported by your course — JUnit5 + TDD approach
- Used in Agile/Scrum teams professionally

9. SDLC & STLC

SDLC — Software Development Lifecycle

- Requirements — gather and document what to build
- System Design — architecture, DB schema, APIs
- Implementation — write the code
- Testing — verify against requirements
- Deployment — release to production
- Maintenance — fix bugs, add features

SDLC Models

- Waterfall — sequential, no going back
- V-Model — testing phase mirrors each dev phase
- Agile — iterative sprints, frequent releases
- Scrum — Agile framework: sprints, backlog, standups
- Spiral — risk-driven, iterative
- Prototype — build mockup first, iterate

STLC — Software Testing Lifecycle

- Requirement Analysis — what needs to be tested
- Test Planning — strategy, scope, resources, timeline
- Test Case Design — write test cases from requirements
- Test Environment Setup — servers, DBs, tools ready
- Test Execution — run tests, log results
- Test Closure — report, lessons learned, metrics

Test Plan vs Test Strategy

- Strategy — high-level, WHAT to test and HOW (org level)
- Plan — specific, WHO, WHEN, WHICH tools (project level)
- Strategy is written once; Plan is per project

10. DEFECT LIFECYCLE & KEY QA TERMS

Defect Lifecycle States

State	Meaning
New	Bug reported for the first time
Assigned	Given to a developer to fix
Open	Developer is actively working on it
Fixed	Developer says it is fixed
Retest	QA re-tests the fix
Verified	QA confirms the fix works
Closed	Bug is resolved and confirmed
Reopened	Bug reappeared after being closed
Rejected	Dev says it is not a bug (by design)
Deferred	Fix postponed to a future release

Duplicate	Same bug already reported

Severity vs Priority

- Severity — impact on system functionality (technical)
- Priority — how urgently it must be fixed (business)

Examples

- High severity, low priority: crash in rarely used feature
- Low severity, high priority: CEO name spelled wrong on homepage
- High severity, high priority: login broken in production

Key QA Terms

- Test Case — steps + expected result for one scenario
- Test Scenario — high-level 'what to test'
- Test Suite — collection of related test cases
- Test Script — automated test code
- Test Bed — environment: hardware, OS, DB, config
- RTM — Requirement Traceability Matrix
- Build — internal release to QA for testing
- Release — external delivery to client/production

Requirement Traceability Matrix (RTM)

RTM maps every requirement to one or more test cases. Ensures no requirement is untested. You can reference this in your interview — 'In my Software Engineering course I learned to map BRS requirements to test cases, which is exactly what an RTM does.'

11. QA vs QC & 7 PRINCIPLES OF TESTING

QA — Quality Assurance

Process-oriented. Prevent defects from occurring.

- Audits, process reviews, standards
- Happens throughout the project
- Focus: are we doing the right process?

QC — Quality Control

Product-oriented. Find defects in the product.

- Testing, code reviews, inspections
- Happens after something is built
- Focus: is this product correct?

7 Principles of Software Testing

- 1. Testing shows presence of defects, not absence
- 2. Exhaustive testing is impossible
- 3. Early testing saves time and cost
- 4. Defects cluster — 80% bugs in 20% of code
- 5. Pesticide paradox — same tests stop finding bugs
- 6. Testing is context-dependent
- 7. Absence of errors fallacy — passing all tests does not mean the product is useful

Verification vs Validation

	Verification	Validation
Question	Are we building the product RIGHT?	Are we building the RIGHT product?
Focus	Process, documents, design	Working software vs real requirements
When	During development	After build, against user needs
Methods	Reviews, inspections, walkthroughs	Testing, UAT, demos

12. INTEGRATION TESTING & STATE MACHINES

Integration Testing

Tests interaction between two or more modules that have been individually unit tested.

Approaches

- Big Bang — integrate all at once, test together
- Top-Down — start from top module, stubs for lower
- Bottom-Up — start from lower modules, drivers for upper

State Machine Testing

Test software whose behaviour depends on STATE — e.g. login states, order status.

- State — a condition the system is in
- Transition — moving from one state to another
- Event — what triggers the transition
- Action — what happens during transition

• Sandwich — combine top-down and bottom-up

Call Graph

Graph showing which modules call which. Used to identify integration order and dependencies.

Example — Login System

• States: Logged Out, Logging In, Logged In, Locked

• Event: correct password -> transition to Logged In

• Event: 3 wrong passwords -> transition to Locked

13. LIKELY INTERVIEW Q&A; — SAY THIS

Q: Tell me about yourself.

A: Final-year CS student at FAST-NUCES graduating June 2026. Studied Software Testing formally under Dr. Usman Ghous — covering TDD, white-box/black-box testing, coverage criteria. Built automation frameworks using JUnit5, Cypress, Mocha/Chai and set up Jenkins CI pipelines. Looking to apply this foundation professionally.

Q: What is the difference between smoke and sanity testing?

A: Smoke: quick check on a new build — is the build stable enough to test? Tests core functionality. Day 0 check. Sanity: after a specific bug fix — does that fix work without breaking nearby functionality? Subset of regression testing.

Q: What is regression testing?

A: Re-running the full test suite after any code change to ensure existing functionality still works. This is where automation pays off most — running 500 tests in minutes instead of manually re-testing everything.

Q: What is the difference between severity and priority?

A: Severity is how badly the bug affects system functionality — technical impact. Priority is how urgently it must be fixed — business impact. Example: a typo on the CEO's name on the homepage is low severity but high priority.

Q: What is TDD?

A: Test Driven Development. Write a failing test first (Red), write minimum code to pass it (Green), then refactor cleanly (Refactor). Ensures every feature has a test by design and forces you to think about requirements before implementation.

Q: What is prime path coverage?

A: A prime path is a maximal simple path in the CFG — no repeated nodes and cannot be extended further. Prime path coverage requires every prime path to be toured. It subsumes edge-pair, edge, and node coverage — our course called it the 'one ring to rule them all'.

Q: What is MCDC / Active Clause Coverage?

A: Each clause in a predicate must independently affect the predicate outcome. CACC variant is closest to MCDC — minor clauses are set so the predicate is true when major clause is true and false when it is false. Used in safety-critical systems like aviation.

Q: What is equivalence partitioning?

A: Divide input domain into partitions where all values behave the same. Test one representative value from each partition. Reduces test cases while maintaining meaningful coverage.

Q: What is boundary value analysis?

A: Test at the edges of partitions — just below minimum, at minimum, just above minimum, just below maximum, at maximum, just above maximum. Bugs concentrate at boundaries.

Q: What is a defect lifecycle?

A: New -> Assigned -> Open -> Fixed -> Retest -> Verified -> Closed. Can also go to Reopened, Rejected, Deferred, or Duplicate depending on outcome.

Q: What is the difference between verification and validation?

A: Verification: are we building the product right? — reviews, inspections against specs. Validation: are we building the right product? — testing against actual user needs. Verification is internal; validation involves the end product against requirements.

Q: Why automate tests?

A: Regression testing manually for every code change is too slow and error-prone. Automation runs hundreds of tests in minutes, provides consistent results, enables CI/CD, and frees QA to focus on exploratory and new feature testing.

Q: What is RTM?

A: Requirement Traceability Matrix. Maps each requirement to test cases. Ensures no requirement is left untested and provides full coverage visibility to stakeholders.

14. HOW TO CONNECT YOUR PROJECTS TO SQA

Project / Experience	QA Angle to Mention
CS3046 Software Testing course	Formal training in TDD, JUnit5, black-box/white-box criteria, logic coverage, integration testing — full semester under Dr. Usman Ghous
JUnit5 Data-Driven Framework	Parameterized tests with CSV data, 84%+ JaCoCo coverage, parallel execution via Maven — real framework built from course knowledge
Node.js AuthApp CI/CD	Mocha/Chai test suite, Jenkins pipeline — automated execution, HTML reports — applied CI/CD principles to testing workflow
RentEase — Cypress POM Suite	End-to-end automation covering auth, property, payment flows using Page Object Model — applied both course concepts and industry patterns
Software Engineering course	SDLC, BRS/SRS, requirements analysis — foundation for test planning, RTM, and understanding what to test
Software Design & Analysis	Design patterns — POM is itself a design pattern. UML — state machines directly map to state-based testing taught in Week 15 of your course
Digital Optimisers internship	REST API testing with Postman, Agile workflow participation, request/response validation — real-world QA exposure
CGPA 3.30 / HEC top 2.3%	Shows academic ability to learn complex testing theory while also building practical projects